

Atividade Prática

Construindo um Encurtador de URLs com **Bun** e **TypeScript**

Duração	3 horas de aula
Modalidade	Individual ou em duplas
Stack	Bun · TypeScript · SQLite (embutido) · HTML/JS
Pré-requisitos	Bun ≥ 1.1, editor de código, navegador, terminal
Entrega	Repositório Git público com README e scripts

Contextualização

Encurtadores de URL (como bit.ly e tinyurl) são serviços web clássicos que ilustram muito bem os fundamentos do backend moderno: rotas HTTP, persistência, geração de identificadores únicos, redirecionamentos e uma interface simples para o usuário final. Nesta atividade, vocês irão construir um encurtador funcional do zero usando o **Bun**, um runtime JavaScript/TypeScript muito mais rápido que o Node.js e que vem com **servidor HTTP**, **SQLite**, **test runner** e **bundler** embutidos — sem precisar instalar Express, better-sqlite3, ts-node, nodemon ou similares.

Objetivos de Aprendizagem

Ao final desta atividade, você deverá ser capaz de:

- Configurar um projeto TypeScript do zero usando o runtime Bun.
- Implementar um servidor HTTP com `Bun.serve()` e roteamento manual.
- Persistir dados em SQLite usando a API nativa `bun:sqlite`.
- Aplicar tipagem estática com TypeScript em modelos de dados e respostas HTTP.
- Validar entradas, tratar erros e retornar respostas JSON apropriadas.
- Construir um frontend HTML simples que consome a sua própria API.

Preparação do Ambiente

Antes de começar, verifique se o Bun está instalado executando `bun --version`. Caso ainda não tenha:

```
# Linux / macOS
curl -fsSL https://bun.sh/install | bash

# Windows (PowerShell)
powershell -c "irm bun.sh/install.ps1 | iex"

# Verifique a instalação
bun --version
```

Dica: o Bun executa arquivos `.ts` diretamente, sem etapa de transpilação. Não é necessário instalar TypeScript ou ts-node.

Parte 1 — Inicializando o Projeto (30 min)

Passo 1.1 — Criar o projeto

Abra o terminal, crie a pasta do projeto e inicialize com o Bun:

```
mkdir encurtador-urls
cd encurtador-urls
bun init -y
```

O comando `bun init -y` cria automaticamente `package.json`, `tsconfig.json`, `index.ts` e `.gitignore`. Apague o conteúdo de `index.ts` — vamos escrever do zero.

Passo 1.2 — Primeiro servidor HTTP

Edite `index.ts` com o servidor mais simples possível:

```
// index.ts
const servidor = Bun.serve({
  port: 3000,
  fetch(req: Request): Response {
    return new Response("Olá, Bun + TypeScript!");
  },
});

console.log(`Servidor rodando em http://localhost:${servidor.port}`);
```

Execute em modo de desenvolvimento (recarrega automaticamente ao salvar):

```
bun --hot index.ts
```

Abra `http://localhost:3000` no navegador. Você deverá ver a mensagem. Tente alterar o texto e salvar — perceba que o servidor atualiza sozinho graças à flag `--hot`.

Reflexão: compare a quantidade de código com o equivalente em Node.js + Express. Quantas dependências do `node_modules` você economizou?

Parte 2 — Banco de Dados com SQLite (30 min)

O Bun traz um cliente SQLite nativo, sem necessidade de drivers externos. Vamos criar um arquivo separado para isolar a camada de dados.

Passo 2.1 — Tipos do domínio

Crie o arquivo `tipos.ts` para concentrar os tipos do projeto:

```
// tipos.ts
export interface UrlEncurtada {
  id: number;
  codigo: string;      // ex.: "aB3xK9"
  urlOriginal: string; // URL completa
  acessos: number;
  criadoEm: string;    // ISO datetime
}

export interface RespostaErro {
  erro: string;
}
```

Passo 2.2 — Camada de banco de dados

Crie `banco.ts`:

```
// banco.ts
import { Database } from "bun:sqlite";
import type { UrlEncurtada } from "./tipos";

const db = new Database("urls.db");

db.exec(`
  CREATE TABLE IF NOT EXISTS urls (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    codigo      TEXT NOT NULL UNIQUE,
    urlOriginal TEXT NOT NULL,
    acessos     INTEGER NOT NULL DEFAULT 0,
    criadoEm    TEXT NOT NULL DEFAULT (datetime('now'))
  );
`);

export function inserirUrl(codigo: string, urlOriginal: string): UrlEncurtada {
  const stmt = db.prepare(
    `INSERT INTO urls (codigo, urlOriginal) VALUES (?, ?)
    RETURNING id, codigo, urlOriginal, acessos, criadoEm`
  );
  return stmt.get(codigo, urlOriginal) as UrlEncurtada;
}

export function buscarPorCodigo(codigo: string): UrlEncurtada | null {
  const stmt = db.prepare("SELECT * FROM urls WHERE codigo = ?");
  return (stmt.get(codigo) as UrlEncurtada) ?? null;
}

export function registrarAcesso(codigo: string): void {
  db.prepare("UPDATE urls SET acessos = acessos + 1 WHERE codigo = ?")
    .run(codigo);
}

export function listarUrls(): UrlEncurtada[] {
  return db.prepare("SELECT * FROM urls ORDER BY id DESC")
    .all() as UrlEncurtada[];
}
```

Atenção: repare no uso de *prepared statements* com ?. Eles são essenciais para evitar **SQL Injection**. Concatenar strings com input do usuário em SQL é uma falha de segurança grave.

Parte 3 — Construindo a API HTTP (45 min)

Passo 3.1 — Função auxiliar para gerar códigos

Crie `util.ts`:

```
// util.ts
const ALFABETO = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";

export function gerarCodigo(tamanho: number = 6): string {
  let codigo = "";
  for (let i = 0; i < tamanho; i++) {
    const indice = Math.floor(Math.random() * ALFABETO.length);
    codigo += ALFABETO[indice];
  }
  return codigo;
}

export function urlValida(url: string): boolean {
  try {
    const u = new URL(url);
    return u.protocol === "http:" || u.protocol === "https:";
  } catch {
    return false;
  }
}
```

Passo 3.2 — Reescrevendo o servidor principal

Substitua o conteúdo de `index.ts` pelo roteador completo:

```
// index.ts
import {
  inserirUrl, buscarPorCodigo, registrarAcesso, listarUrls
} from "./banco";
import { gerarCodigo, urlValida } from "./util";
import type { RespostaErro } from "./tipos";

function json(dados: unknown, status: number = 200): Response {
  return new Response(JSON.stringify(dados, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8" },
  });
}

function erro(mensagem: string, status: number): Response {
  const resp: RespostaErro = { erro: mensagem };
  return json(resp, status);
}
```

Agora adicione o servidor com as rotas. Continue no mesmo arquivo:

```
const servidor = Bun.serve({
  port: 3000,

  async fetch(req: Request): Promise<Response> {
    const url = new URL(req.url);
    const caminho = url.pathname;
    const metodo = req.method;

    // POST /api/encurtar -> cria nova URL curta
    if (metodo === "POST" && caminho === "/api/encurtar") {
      const corpo = await req.json() as { urlOriginal?: string };
      if (!corpo.urlOriginal || !urlValida(corpo.urlOriginal)) {
        return erro("URL inválida. Forneça http:// ou https://", 400);
      }
      const codigo = gerarCodigo();
      const registro = inserirUrl(codigo, corpo.urlOriginal);
      return json(registro, 201);
    }

    // GET /api/urls -> lista todas
    if (metodo === "GET" && caminho === "/api/urls") {
      return json(listarUrls());
    }

    // GET /:codigo -> redireciona
    if (metodo === "GET" && /^\/[A-Za-z0-9]{4,10}$/.test(caminho)) {
      const codigo = caminho.slice(1);
      const registro = buscarPorCodigo(codigo);
      if (!registro) return erro("Código não encontrado", 404);
      registrarAcesso(codigo);
      return Response.redirect(registro.urlOriginal, 302);
    }

    // GET / -> serve o frontend
    if (metodo === "GET" && caminho === "/") {
      return new Response(Bun.file("./public/index.html"));
    }

    return erro("Rota não encontrada", 404);
  },
});

console.log(`■ Servidor pronto em http://localhost:${servidor.port}`);
```

Passo 3.3 — Testando com curl

Com o servidor rodando (`bun --hot index.ts`), abra outro terminal e teste:

```
# Criar uma URL curta
curl -X POST http://localhost:3000/api/encurtar \
  -H "Content-Type: application/json" \
  -d '{"urlOriginal": "https://www.ifes.edu.br"}'

# Listar todas
curl http://localhost:3000/api/urls

# Redirecionar (use o código retornado, ex.: aB3xK9)
curl -L http://localhost:3000/aB3xK9
```


Parte 4 — Frontend Simples (45 min)

Vamos criar uma página HTML que consome a API. Crie a pasta `public/` e dentro dela o arquivo `index.html`:

```
<!-- public/index.html -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Encurtador de URLs</title>
  <style>
    body { font-family: system-ui; max-width: 600px;
      margin: 40px auto; padding: 0 20px; }
    h1 { color: #0d4f3c; }
    input, button { font-size: 16px; padding: 10px; }
    input { width: 70%; }
    button { background: #0d4f3c; color: white;
      border: none; cursor: pointer; }
    .resultado { margin-top: 20px; padding: 12px;
      background: #eef6f3; border-left: 4px solid #1a8870; }
    table { width: 100%; border-collapse: collapse; margin-top: 30px; }
    th, td { padding: 8px; border-bottom: 1px solid #ddd;
      text-align: left; }
    th { background: #0d4f3c; color: white; }
  </style>
</head>
<body>
  <h1>■ Encurtador de URLs</h1>
  <input id="url" type="url" placeholder="https://exemplo.com/...">
  <button id="btn">Encurtar</button>
  <div id="resultado"></div>
  <h2>URLs cadastradas</h2>
  <table id="tabela">
    <thead><tr><th>Código</th><th>Original</th><th>Acessos</th></tr></thead>
    <tbody></tbody>
  </table>
  <script src="app.js"></script>
</body>
</html>
```

Agora crie `public/app.js` para interagir com a API:

```
// public/app.js
const btn = document.getElementById("btn");
const input = document.getElementById("url");
const resultado = document.getElementById("resultado");
const tbody = document.querySelector("#tabela tbody");

async function carregarLista() {
  const resp = await fetch("/api/urls");
  const lista = await resp.json();
  tbody.innerHTML = lista.map(u => `
    <tr>
      <td><a href="/${u.codigo}" target="_blank">${u.codigo}</a></td>
      <td>${u.urlOriginal}</td>
      <td>${u.acessos}</td>
    </tr>`).join("");
}

btn.addEventListener("click", async () => {
  const resp = await fetch("/api/encurtar", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ urlOriginal: input.value }),
  });
  const dados = await resp.json();
  if (!resp.ok) {
    resultado.innerHTML = `<div class="resultado">■ ${dados.erro}</div>`;
    return;
  }
  const curta = `${location.origin}/${dados.codigo}`;
  resultado.innerHTML = `
    <div class="resultado">
      ■ URL curta: <a href="${curta}" target="_blank">${curta}</a>
    </div>`;
  input.value = "";
  carregarLista();
});

carregarLista();
```

Falta apenas servir o `app.js`. Adicione esta rota no `index.ts`, antes do `return erro(...404)`:

```
if (metodo === "GET" && caminho === "/app.js") {
  return new Response(Bun.file("./public/app.js"));
}
```

Checkpoint: acesse `http://localhost:3000`. Você deve conseguir colar uma URL, gerar um código curto, ver a lista atualizar e clicar no código para ser redirecionado. Mostre o resultado ao professor antes de prosseguir para os desafios.

Parte 5 — Desafios de Extensão (30 min)

Escolha **pelo menos dois** dos desafios abaixo e implemente. Eles compõem parte da nota da atividade.

Desafio A — Códigos personalizados

Permita que o usuário informe opcionalmente um código no corpo da requisição POST. Valide formato (apenas letras e números, 4–10 caracteres) e devolva 409 `Conflict` se já existir.

Desafio B — Página de estatísticas

Crie a rota GET `/stats/:codigo` que retorna um HTML com informações da URL: original, data de criação, quantidade de acessos e QR Code (pode usar a API pública `api.qrserver.com`).

Desafio C — Expiração de links

Adicione uma coluna `expiraEm` (opcional). Quando preenchida, ao tentar acessar um link expirado, retorne 410 `Gone` em vez de redirecionar.

Desafio D — Testes automatizados

Use o test runner embutido do Bun para cobrir as funções de `util.ts` e `banco.ts`:

```
// util.test.ts
import { expect, test, describe } from "bun:test";
import { gerarCodigo, urlValida } from "../util";

describe("gerarCodigo", () => {
  test("gera com 6 caracteres por padrão", () => {
    expect(gerarCodigo()).toHaveLength(6);
  });
});

describe("urlValida", () => {
  test("aceita http e https", () => {
    expect(urlValida("https://ifes.edu.br")).toBe(true);
  });
  test("rejeita string aleatória", () => {
    expect(urlValida("abc")).toBe(false);
  });
});
```

Execute com `bun test`.

Desafio E — Variáveis de ambiente

Crie um arquivo `.env` com `PORTA=3000` e leia com `Bun.env.PORTA`. Documente no README como alterar.

Critérios de Avaliação

Critério	Peso
Servidor funcional com todas as rotas obrigatórias (Partes 1–3)	30%

Persistência correta em SQLite e uso de prepared statements	20%
Frontend integrado e funcional (Parte 4)	20%
Implementação de pelo menos 2 desafios da Parte 5	20%
Organização do código, README e commits no Git	10%

Entrega

Envie o link de um repositório **público** no GitHub contendo o código completo, um README.md com instruções de execução (`bun install` e `bun --hot index.ts`) e um histórico de commits com mensagens descritivas. Bons commits valem nota.

Bons estudos! Lembrem-se: dúvidas pontuais durante a aula valem mais do que horas depois travados em casa. Chamem o professor.